

Communauté Française de Belgique

Institut des Carrières Commerciales

Ville de Bruxelles

Waterside

Quai de Willebroeck, 33

1000 Bruxelles



**Rapport
d'analyse UML 2
Version 3/3**

Tarik EL MAHTOUCI

2025-2026

Table des matières

1	Description	3
1.1	Diagramme de cas d'utilisation	3
1.2	Diagramme de classe	3
1.3	Diagramme de séquence	3
1.4	Diagramme de navigation d'activité	3
1.5	Diagramme d'états-transitions	3
2	Logiciel utilisé	3
2.1	Bibliothèque de formes UML	3
2.2	Personnalisation avancée	4
3	Normes UML 2	4
4	Modélisation visuelle	4
4.1	Diagramme de cas d'utilisation	4
4.2	Diagramme de classe	5
4.3	Diagramme de séquence	5
4.4	Diagramme de navigation	6
4.5	Diagramme d'activité	7
4.6	Diagramme d'états-transition (réservation)	7
5	Design patterns utilisés	8
5.1	Factory Method	8
5.2	Singleton	8
5.3	Strategy	8
5.4	Observer	8
5.5	Façade	8
5.6	Model-View-Controller (MVC)	8

1. Description

Les diagrammes UML du projet **BikeShare** offrent une vue complète et diversifiée du système de location de vélos entre particuliers. Ils sont essentiels pour modéliser et comprendre les différents aspects du système.

1.1 Diagramme de cas d'utilisation

Ce diagramme identifie les interactions entre les acteurs (Visiteur, Membre, Administrateur) et le système. Il met en évidence les fonctionnalités accessibles : recherche et réservation de vélos, gestion des comptes, communication et traitement des paiements.

1.2 Diagramme de classe

Il expose la structure statique du système : classes principales, attributs, méthodes et relations (associations, cardinalités). Il clarifie la conception des objets et leurs interactions.

1.3 Diagramme de séquence

Il montre la séquence chronologique des interactions entre objets, notamment le flux de réservation d'un vélo, de la demande initiale jusqu'à la confirmation et le paiement.

1.4 Diagramme de navigation d'activité

Il représente les flux de travail au sein du système : enchaînement des pages, décisions et flux de données lors du parcours de réservation.

1.5 Diagramme d'états-transitions

Il décrit les états possibles d'une réservation et les transitions entre ces états selon les événements (demande, acceptation, paiement, annulation, clôture).

2. Logiciel utilisé

Les diagrammes UML de BikeShare ont été réalisés avec **draw.io**, un logiciel en ligne reconnu pour sa convivialité et sa flexibilité, permettant de concevoir des diagrammes complexes de manière intuitive.

2.1 Bibliothèque de formes UML

Une vaste collection de formes conformes aux normes UML facilite la représentation précise des composants : diagrammes

de classe, de cas d'utilisation, de séquence, d'états-transitions et d'activités.

2.2 Personnalisation avancée

L'outil offre des options de personnalisation (couleurs, polices, tailles, styles de connexions, annotations) pour une présentation visuelle claire et adaptée au projet.

3. Normes UML 2

Les diagrammes suivent les conventions de la norme **UML 2**, assurant une représentation claire et standardisée des éléments et interactions du système. Cela facilite la communication des concepts techniques aux parties prenantes et la documentation des aspects structurels et comportementaux de BikeShare.

4. Modélisation visuelle

Cette section présente les diagrammes UML modélisant les aspects structurels et comportementaux du système BikeShare.

4.1 Diagramme de cas d'utilisation

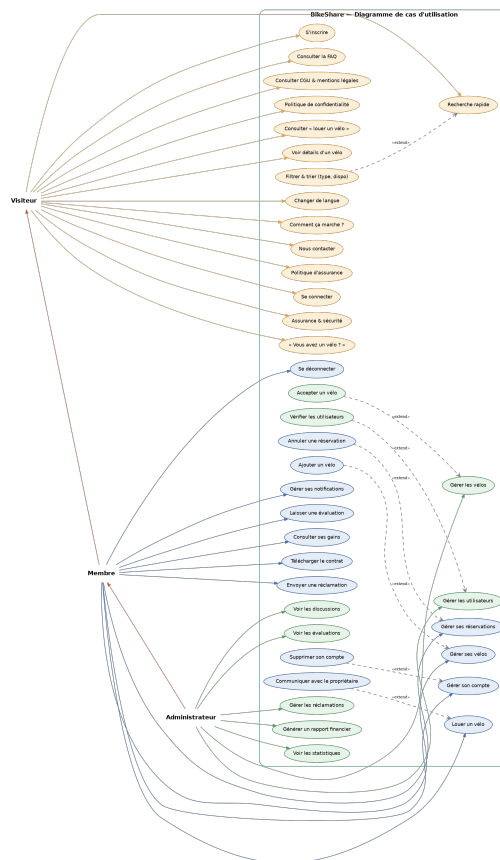


Diagramme de cas d'utilisation — BikeShare

4.2 Diagramme de classe

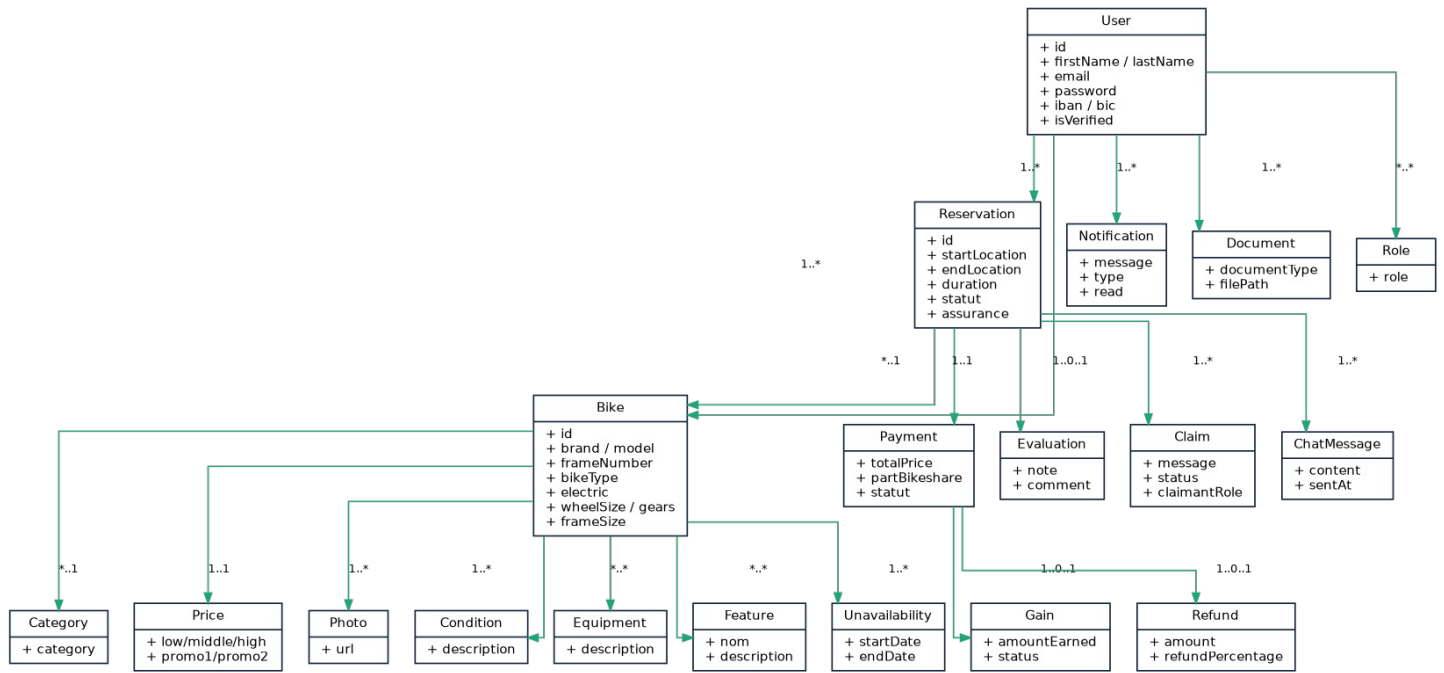


Diagramme de classes — BikeShare

4.3 Diagramme de séquence

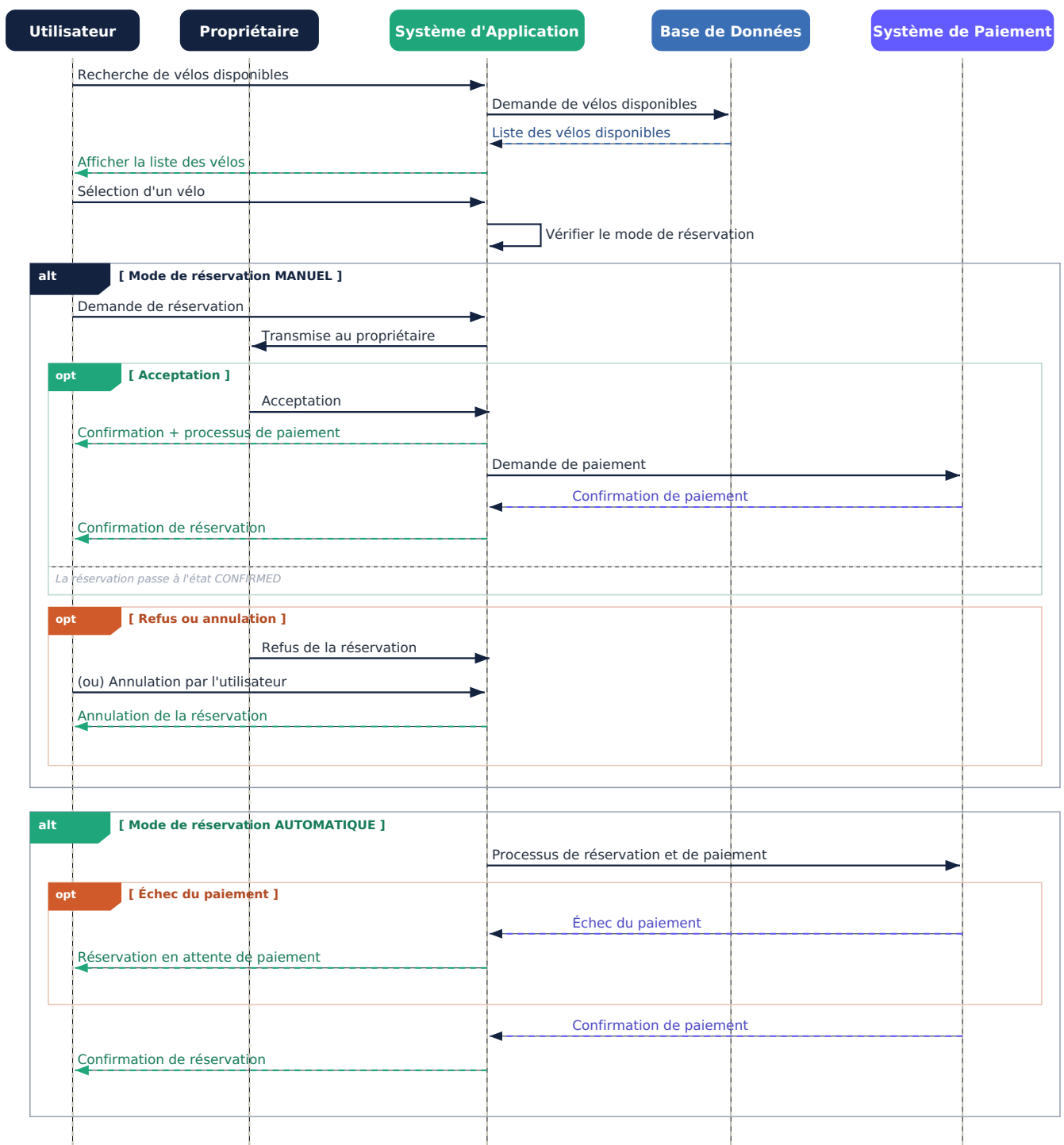


Diagramme de séquence — réservation et paiement

4.4 Diagramme de navigation

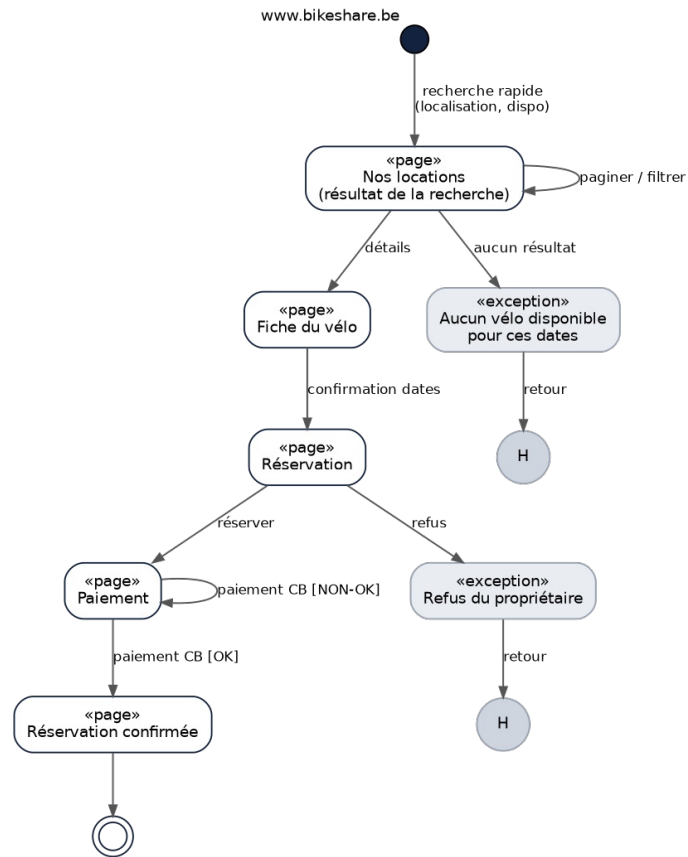


Diagramme de navigation — BikeShare

4.5 Diagramme d'activité

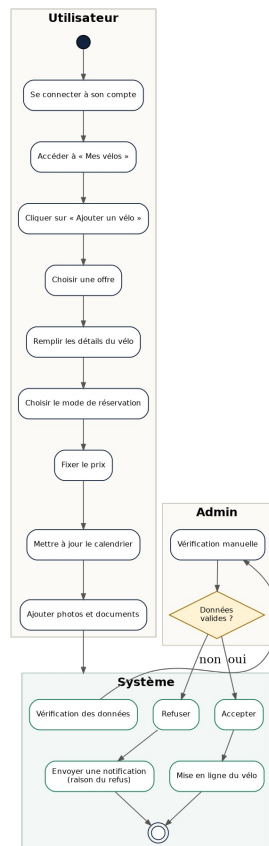


Diagramme d'activité — publication d'un vélo

4.6 Diagramme d'états-transition (réservation)

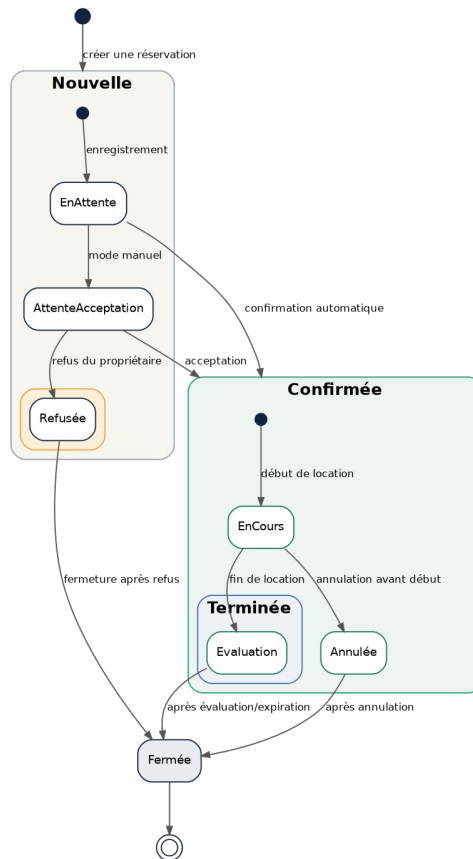


Diagramme d'états-transition — réservation

5. Design patterns utilisés

Pour une application de location de vélos entre particuliers comme BikeShare, l'utilisation de patrons de conception structure le code, améliore la maintenabilité et l'extensibilité, et gère les opérations complexes.

5.1 Factory Method

Gère la création d'objets de façon flexible. *Application* : instancier différents types de vélos (ville, VTT, électrique, cargo) selon les données d'entrée.

5.2 Singleton

Assure une instance unique avec un point d'accès global. *Application* : centraliser les configurations de l'application BikeShare.

5.3 Strategy

Permet de varier l'algorithme utilisé. *Application* : diverses stratégies de tarification (durée, saison, promotions).

5.4 Observer

Notifie une liste d'observateurs des changements d'état. *Application* : notifier les utilisateurs des changements de statut de réservation ou des nouveaux messages.

5.5 Façade

Simplifie l'interface d'un sous-système. *Application* : la couche service masque la complexité des opérations (réserver, annuler, payer, évaluer).

5.6 Model-View-Controller (MVC)

Sépare l'application en modèle, vue et contrôleur. *Application* : modèle (entités JPA et logique métier), vue (Thymeleaf), contrôleur (gestion des interactions). L'intégration de ces patrons assure une base solide, facilitant l'évolution et la maintenance de BikeShare.